

Unidad 2

Generación de código intermedio

Del AST validado a **instrucciones simples** (IR)

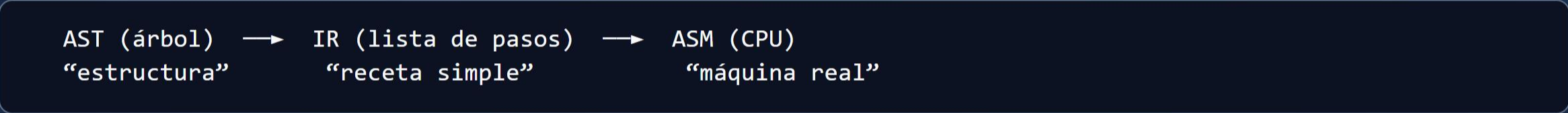
Glosario Unidad 2

Término	Significado
IR	Representación intermedia — “lenguaje puente” del compilador
Triplo	Instrucción de 4 partes: operador, arg1, arg2, resultado
Temporal (t1, t2...)	Variable ficticia que inventa el compilador
Postfija	Notación operador al final: <code>3 4 +</code> en lugar de <code>3 + 4</code>
Postorden	Recorrer AST: hijos primero, raíz al final
Label (L1, L2)	Etiqueta destino de un salto (<code>if</code> , <code>goto</code>)
Back-end	Parte que traduce IR hacia la máquina (U3–U4)

IR no es ensamblador. No verás `mov eax` todavía — eso es Unidad 4.

¿Qué es IR?

IR = *Intermediate Representation* = **representación intermedia**



No es IR	Sí es IR	
Código MiniLang	<code>t1 = 4 * 2</code>	
Comentarios del alumno	<code>ifFalse t1 goto L1</code>	
Tabla de símbolos	Triplos numerados	

¿Por qué no ir directo AST → ASM?

Ventaja de usar IR

Instrucciones uniformes → fácil de optimizar (U3)

Mismo front-end, distinta CPU (x86, ARM...)

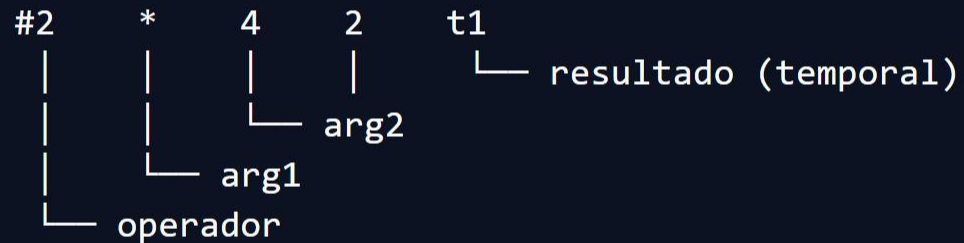
Separar entender el programa de traducirlo

El optimizador trabaja **solo sobre IR**, no sobre texto fuente ni ASM.

Formatos de IR en el curso

Formato	Ejemplo	Uso en curso
Postfija	3 4 2 * +	Entender orden de evaluación
Triplos	(+, 3, t1, t2)	Formato principal
Cuádruplos	(#2, +, 3, t1, t2)	Igual + número de línea #
Código P	pila: ldc 3, add	Referencia opcional

Anatomía de un triplo



Significa en español: «**multiplica 4 por 2 y guarda en t1**».

Ejemplo completo: $x = 3 + 4 * 2$

#	Operador	arg1	arg2	resultado
1	*	4	2	t1
2	+	3	t1	t2
3	=	t2	—	x

Orden respeta **precedencia**: * antes que +, igual que en matemáticas.

Postfija — ¿cómo se obtiene?

Expresión `a + b * c`:

1. Recorrer AST en **postorden** (hijos → padre)
2. Operando → apilar
3. Operador → desapilar 2, calcular, apilar resultado

Resultado: `a b c * +`

(Lee: `a + (b * c)`)

Postfija en pizarra

$3 + 4 * 2 \rightarrow$ postfija: $3 4 2 * +$

Evaluar con **pila**:

push 3 $\rightarrow$ push 4 $\rightarrow$ push 2 $\rightarrow$ * $\rightarrow$ queda 8 $\rightarrow$ + $\rightarrow$ queda 11

Postfija = misma expresión **sin paréntesis** porque el orden de la pila respeta precedencia.

Temporales t1, t2, t3...

```
int x = 3 + 4 * 2;  // el programador solo escribe "x"
```

Internamente el compilador crea:

```
t1 = 4 * 2  
t2 = 3 + t1  
x  = t2
```

Temporales = cajas de trabajo; **no** aparecen en tu código fuente.

Control de flujo — if/else

```
if (x > 5) { x = x - 1; } else { x = 0; }
```

Triplos típicos:

```
t1 = (x > 5)           ; comparación
iffalse t1 goto L1     ; si es falso, saltar al else
x = x - 1              ; rama then
goto L2                ; saltar al fin
L1: x = 0               ; rama else
L2:                    ; continuación
```

Labels y saltos — vocabulario

Instrucción IR	Significado en español
<code>ifFalse t, L1</code>	Si <code>t</code> es falso, salta a etiqueta L1
<code>goto L2</code>	Salto incondicional a L2
<code>L1:</code>	Aquí continúa quien saltó

En Unidad 4 esto se convierte en `cmp`, `je`, `jmp` (ensamblador).

Cuádruplo vs triplo

	Triplo	Cuádruplo
Campos	op, arg1, arg2, res	# , op, arg1, arg2, res
Índice	opcional	número de instrucción fijo
En el curso	Generación principal	Reconocer en examen

Ambos describen **la misma operación**; cambia el formato tabular.

Errores comunes

1. Sumar **antes** de multiplicar (ignorar precedencia)
2. Olvidar un **temporal** en cadena de operaciones
3. Poner etiqueta **L1** antes del código del *then*
4. Confundir IR con ASM (`mov eax` **no** es IR)
5. Mezclar **tabla de símbolos** con archivo `.ir`

Entregable Unidad 2

```
compilador/ir/  
  generator.py  
ejemplos/  
  aritmetica.ir  
  if_else.ir
```

Formato **.ir**:

```
#1  (*, 4, 2, t1)  
#2  (+, 3, t1, t2)  
#3  (=, t2, , x)
```

Repaso — 5 ideas

1. **IR** = puente entre AST y CPU
2. **Triplos** = operador + operandos + resultado
3. **Postfija** = recorrido postorden del AST
4. **Temporales** t1, t2... son del compilador
5. **if/else** = comparar + `ifFalse` + `goto` + labels

Próxima: optimizar IR sin cambiar el resultado